

MixBytes()

[SECURITY AUDIT REPORT]

Mellow Finance SyncRedeemQueue

PREPARED FOR LIDO

REPORT DATE

July 15, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	11
2.1 Critical	11
2.2 High	11
2.3 Medium	12
M-1 Synchronous redemption at a stale oracle price enables stale-price arbitrage / front-running	12
M-2 Synchronous redemption ignores outstanding async redeem demand and can jump ahead of finalized async requests	14
M-3 Synchronous redemption loses accrued protocol fee on the burned shares	15
2.4 Low	17
L-1 Global daily-limit bucket allows one holder to transiently block all redemptions	17
L-2 redeem has no user-side slippage protection while execution terms can change between signing and inclusion	18
L-3 dailyLimit validation footguns: % 24 hours rejects natural values, dailyLimit = 0 silently bricks the queue, and lowering the limit freezes it	19
L-4 penaltyD6 cap of 5e5 (50%) is excessively generous for a parameter that directly confiscates user value	20
L-5 ETH sent directly to the queue (or a redemption with receiver == address(this)) is permanently stuck - no rescue method	21
L-6 Under a prolonged oracle outage there is no exit path, and recovery requires two distinct oracle roles	22
L-7 modifyVaultBalance is called after the external asset transfer in redeem (checks-effects-interactions)	23
L-8 maxAge has no lower bound tied to the oracle report cadence, so a below-timeout value causes a recurring self-inflicted DoS	24
3. About MixBytes	25

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

`SyncRedeemQueue` is the synchronous redemption component of Mellow's Flexible Vaults, a modular vault framework in which share issuance, redemption, pricing, fees, and risk limits are split across dedicated modules and pluggable queues. Unlike the asynchronous `RedeemQueue`, which settles requests against a future oracle report, this queue lets a shareholder burn shares and receive the underlying asset instantly in a single transaction, priced against the latest stored oracle report. Redemptions apply a configurable penalty and redeem fee, are throttled by a per-queue leaky-bucket daily limit, and are constrained by the vault's available liquidity and risk-manager accounting.

The interim security review was conducted over 2 working days via manual code review and a proprietary AI-assisted analysis tool.

During the audit, our focus covered both the core redemption logic of `SyncRedeemQueue` and its on-chain interactions with the surrounding vault modules (oracle pricing, risk-manager balance accounting, fee and penalty handling, share burning/minting, and redeem hooks). Alongside these, the team worked through the standard internal checklist, including access control, reentrancy and cross-contract reentrancy, arithmetic overflow, rounding and precision loss. Beyond these baseline classes, the following protocol-specific attack vectors were examined:

- **Stale-price redemption.** Whether the instant, single-report pricing lets a redeemer transact at a self-selected stale price to claim more assets than a fresh price would allow, given `maxAge` decoupled from oracle cadence.
- **Daily-limit accounting.** Whether the leaky-bucket decay, ordering of the usage check and update, and parameter changes can be manipulated to redeem beyond the intended cap, or to block others by exhausting the shared global bucket.
- **Penalty, fee, and payout arithmetic.** Whether rounding across penalty, redeem fee, and asset conversion, together with the full-share burn versus asset-based risk-manager debit, lets a redeemer extract value or drifts the vault's share/limit accounting.
- **Cross-contract reentrancy on settlement.** Whether the asset transfer to a caller-controlled `receiver` (ETH / callback tokens) before the risk-manager balance update lets an attacker re-enter other vault flows while liquidity and balances are inconsistent.
- **Instantly-timed subvault drain.** When vault balance is short, `BasicRedeemHook.callHook` pulls from subvault loose balances; unlike the staged async path, sync `redeem` lets any holder force an immediate pull of up to `dailyLimit`-worth of subvault working float, so strategies must not assume loose subvault balances persist within a block.

The `Oracle`, `RiskManager`, `FeeManager`, `ShareManager` / `ShareModule`, and `redeem-hook` contracts that `SyncRedeemQueue` calls into were not part of this engagement's line-by-line scope; our review of them was limited to the integration points reachable from the in-scope redemption path and their observed on-chain behavior.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Well-structured and consistent with the surrounding framework.** The queue follows the established module patterns of the codebase (ERC-7201 namespaced storage via `SlotLibrary`, factory-driven initialization with `_disableInitializers`, role checks delegated to the vault's access control, and `nonReentrant` on the state-mutating entry point). The redemption path is written in a broadly checks-effects-interactions order, and value flows (penalty, fee, payout) are individually traceable.
- **Leaky-bucket, penalty, and fee arithmetic are conservative.** Daily-limit accounting, linear usage decay, and the penalty/fee math round in the protocol's favor and are correctly ordered; the documented burst bound of up to `2 * dailyLimit` per 24h holds. The daily limit is a single global bucket per queue, so one large holder can transiently consume the day's capacity, though only at a real penalty-and-fee cost.
- **Risk-manager balance intentionally diverges from total share supply.** `modifyVaultBalance` converts the asset delta to share-units via `convertToShares` ($\times$ `priceD18`) and accumulates it into `vaultState.balance`, so `balance` is a price-weighted quantity rather than a mirror of `totalShares()`. On redemption the share supply drops by the gross `shares` burned, but `balance` is debited only by the net payout (`sharesToRedeem`), so the penalty and fee portions leave `balance` higher than the supply-implied value — an intended offset (penalty value stays in the vault as assets), which we recommend documenting so integrators do not treat `vaultState.balance` as `totalShares()`.
- **Minor hardening opportunities.** The `receive()` function allows stray ETH to accumulate in the queue with no sweep path, and several supporting vault functions (`callHook`, `getLiquidAssets`, `claimShares`) rely on being reached only through guarded queue entry points and live-balance reads; documenting these invariants would reduce fragility to future changes.

1.3 Project Overview

Summary

CLIENT	Mellow Finance
CATEGORY	Liquid Restaking
PROJECT	Flexible Vaults
TYPE	Solidity
PLATFORM	EVM
TIMELINE	08.07.2026 - 13.07.2026

Scope of Audit

```
src/queues/SyncRedeemQueue.sol
```

```
src/interfaces/queues/ISyncRedeemQueue.sol
```

Versions Log

DATE	COMMIT HASH	NOTE
08.07.2026	69bbaf17530dc5127ecf9049cceaab141f516a1e	Initial Commit

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
SyncRedeemQueue.sol	0x0000000038801C7281284f8F68B80B679F64a074	Ethereum

The deployment was reviewed and its bytecode and configuration match the audited scope. The contract compiles byte-for-byte from the audit-scope source at commit

69bbaf17530dc5127ecf9049cceaab141f516a1e with the pinned dependency (@openzeppelin/contracts-upgradeable v5.3.0 at 60b305a8f3ff0c7688f02ac470417b6bbf1c4d27). Both the creation init-code and the runtime bytecode - including the trailing solc metadata - reproduce identically; the only deviations are the two constructor-immutable slots.

The immutables decode to the deterministic storage slots derived from the constructor arguments name = "Mellow" and version = 1: _syncQueueStorageSlot = 0xdf967b659a92a84e0f5b413833268193250b41d91ce24f46eb30e11839f54f00 (SlotLibrary.getSlot("SyncQueue", "Mellow", 1)) and _syncRedeemQueueStorageSlot = 0xbc5c728f3fd3947225cf056ebbb2c2c47a76b671fa45a433962efd70cc3d4f00 (SlotLibrary.getSlot("SyncRedeemQueue", "Mellow", 1)), both matching the expected values recomputed off-chain. The SET_SYNC_REDEEM_PARAMS_ROLE constant reads back as keccak256("queues.SyncRedeemQueue.SET_SYNC_REDEEM_PARAMS_ROLE") as expected.

This is the queue's logic (implementation) contract, deployed to a vanity address through the canonical CREATE2 deterministic deployer (0x4e59b44847b379578588920cA78FbF26c0B4956C) by EOA 0xE98Be1E5538FCbD716C506052eB1Fd5d6fC495A3 . In this framework queues are never used directly: Factory.create deploys a TransparentUpgradeableProxy over a registered implementation and runs initialize on the proxy, so per-vault instances live in proxy storage while this address only serves as delegatecall logic. Consistent with that role, its initializers are locked - the _disableInitializers() call in the constructor sets the OpenZeppelin Initializable version slot to type(uint64).max, so the implementation cannot be initialized directly and is not exposed to initialization front-running - and it holds no configuration of its own (vault() , asset() , and syncRedeemParams() all read back as zero).

At the time of review the implementation is deployed but not yet in use: it has received no transactions since creation, no Factory has proposed or accepted it as an implementation (no ProposeImplementation / AcceptProposedImplementation events reference it), and no proxy has been created over it. Re-validation will therefore be required once it is put into production - to confirm it is registered under the intended Factory and version, and, for each proxy instance created from it, that the initialize() arguments (asset, share module, and the penaltyD6 / maxAge / dailyLimit sync-redeem parameters) and the SET_SYNC_REDEEM_PARAMS_ROLE grant are configured as intended.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

ID	FINDING	SEVERITY	STATUS
M-1	Synchronous redemption at a stale oracle price enables stale-price arbitrage / front-running	● MEDIUM	● ACKNOWLEDGED
M-2	Synchronous redemption ignores outstanding async redeem demand and can jump ahead of finalized async requests	● MEDIUM	● ACKNOWLEDGED
M-3	Synchronous redemption loses accrued protocol fee on the burned shares	● MEDIUM	● ACKNOWLEDGED
L-1	Global daily-limit bucket allows one holder to transiently block all redemptions	● LOW	● ACKNOWLEDGED
L-2	redeem has no user-side slippage protection while execution terms can change between signing and inclusion	● LOW	● ACKNOWLEDGED
L-3	dailyLimit validation footguns: % 24 hours rejects natural values, dailyLimit = 0 silently bricks the queue, and lowering the limit freezes it	● LOW	● ACKNOWLEDGED
L-4	penaltyD6 cap of 5e5 (50%) is excessively generous for a parameter that directly confiscates user value	● LOW	● ACKNOWLEDGED
L-5	ETH sent directly to the queue (or a redemption with receiver == address(this)) is permanently stuck - no rescue method	● LOW	● ACKNOWLEDGED
L-6	Under a prolonged oracle outage there is no exit path, and recovery requires two distinct oracle roles	● LOW	● ACKNOWLEDGED

ID	FINDING	SEVERITY	STATUS
L-7	modifyVaultBalance is called after the external asset transfer in redeem (checks-effects-interactions)	● LOW	● ACKNOWLEDGED
L-8	maxAge has no lower bound tied to the oracle report cadence, so a below-timeout value causes a recurring self-inflicted DoS	● LOW	● ACKNOWLEDGED

[2 FINDINGS REPORT]

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

M-1 Synchronous redemption at a stale oracle price enables stale-price arbitrage / front-running

● MEDIUM

● ACKNOWLEDGED

DESCRIPTION

`SyncRedeemQueue.redeem` settles instantly and prices the redemption against the currently stored oracle report, gated only by a staleness check:

```
if (report.timestamp + $.maxAge < block.timestamp) {
    revert StaleReport();
}
```

[src/queues/SyncRedeemQueue.sol#L99-L101](#)

Since `assets = mulDiv(sharesToRedeem, 1 ether, priceD18)`, a stored price below the true price overpays the redeemer at remaining holders' expense. `maxAge` is configurable up to `365 days` with no coupling to the oracle's own cadence, so - unlike the `async RedeemQueue`, which ages each request against a future report (`redeemInterval`) - a redeemer can choose the moment of redemption and exit at a self-selected stale price anywhere in the window.

The magnitude depends heavily on the used oracle's `SecurityParams`. The codebase's live configs bracket the range. The stablecoin `earnUSD*` oracles (e.g. [scripts/plasma/earnUSDDe.s.sol#L138-L140](#)) set `maxRelativeDeviationD18 = 0.5%`, `suspiciousRelativeDeviationD18 = 0.1%`, `timeout = 20 h`: a single report cannot move more than 0.5% (a larger move reverts `InvalidPrice`) and any move above 0.1% is stored `isSuspicious` - on which `redeem` reverts `InvalidReport`. Under such gates a sharp real drawdown cannot be arbitrated continuously; the window shrinks to the reaction-latency gap before the oracle's first reaction, bounded per period by `dailyLimit`. The `strETH` oracles ([scripts/plasma/PlasmaStrETH.s.sol#L120-L122](#)) sit at the opposite extreme - `maxRelativeDeviationD18 = suspiciousRelativeDeviationD18 = 1 wei`, `timeout = type(uint32).max` - where those gates provide no bound and the stale window is limited only by `maxAge`. In all cases, because the sync path bypasses `redeemInterval`, `penaltyD6` is the sole protection.

The risk is symmetric across the queue pair: the same stale-price window lets the sibling `SyncDepositQueue` be exploited in the opposite direction - when the true price falls sharply before the oracle updates, a deposit lands at the stale-high price and mints excess shares (a sandwich around the pending report). Both directions share one root cause (a `maxAge`-wide window decoupled from oracle cadence) and one backstop (`penaltyD6`).

Severity is Medium: the path is real but parameter-gated, and its magnitude turns on the paired oracle's deviation gates and `maxAge` sizing.

RECOMMENDATION

We recommend making backstop to be economic, not a tight `maxAge` (with a `20h` timeout, `maxAge` must stay $\geq \sim 24\text{--}30$ h or the queue self-DoSes). Size `penaltyD6` above the paired oracle's maximum single-report deviation (`maxRelativeDeviationD18`) plus margin; size `dailyLimit`

explicitly as the damage cap for the frozen-price drawdown; and lower the 365-day validation cap to slightly above the oracle `timeout` so a stuck oracle cannot leave a multi-month stale window open.

CLIENT'S COMMENTARY

By design.

The 365-day upper bound for `maxAge` is merely a sanity/adequacy check. The actual `maxAge` value is governed by the protocol administrator and is expected to be configured close to `oracle.securityParams().timeout` (approximately one day in our deployment), not anywhere near the theoretical upper limit.

Likewise, `penaltyD6` is expected to remain 0 in normal operation. In practice, users redeem before the next oracle report is published and therefore already receive fewer assets than they would after the updated report is available. Applying an additional penalty in this scenario would unnecessarily penalize users twice.

Additionally, the deployment scripts referenced in the report is not representative of the intended production setup. For both `strETH` and `earnUSDe`, oracle reports are only updated on Ethereum; on all other networks, they are intentionally not updated. Furthermore, deposit and withdrawal queues of any kind are neither deployed nor planned to.

M-2 Synchronous redemption ignores outstanding async redeem demand and can jump ahead of finalized async requests

● MEDIUM

● ACKNOWLEDGED

DESCRIPTION

The sync and async redeem queues share the same vault liquidity pool, but the sync path does not account for liquidity already owed to the async queue. At report time the async `RedeemQueue` burns the redeemer's shares and records a `Batch` with `totalDemandAssets += assets` (`src/queues/RedeemQueue.sol#L239-L243`) - a finalized obligation - and `handleBatches` pays batches in order only while liquidity covers them. `SyncRedeemQueue.redeem` instead checks `assets <= getLiquidAssets()` (`src/queues/SyncRedeemQueue.sol#L117-L120`), and `getLiquidAssets` returns raw vault + subvault balances without subtracting `totalDemandAssets` - so it treats liquidity already promised to finalized async redemptions (whose shares are already burned) as free.

Scenario: vault liquidity = 100. Alice requests an async redeem worth 100; the report burns her shares and records a 100-asset batch (she cannot cancel - async has no cancel). Before a keeper runs `handleBatches`, Bob synchronously redeems 100: `getLiquidAssets` still reports 100, so Bob's call passes and drains the pool. `handleBatches` then finds `demand (100) > liquidAssets (0)` and stalls - Alice is stuck until liquidity replenishes. Notably the deposit side implements exactly the missing reservation - a sync deposit's limit check includes pending async deposits' `pendingBalance` (`src/managers/RiskManager.sol#L229`) - so the redeem-side omission is an oversight relative to the codebase's own pattern. Severity is Medium: gated on unhandled async batches existing at the moment of a sync redeem, no funds lost.

RECOMMENDATION

Make the sync liquidity check reserve outstanding async demand - require `assets <= getLiquidAssets() - totalDemandAssets`, where `totalDemandAssets` is the async `RedeemQueue`'s single running counter of finalized-but-unpaid obligations. In the standard topology (one async redeem queue per asset) this is one value; since `ShareModule` does not cap redeem queues at one per asset, the general form subtracts every redeem queue's `totalDemandAssets` for the asset (or `getLiquidAssets` itself aggregates it).

CLIENT'S COMMENTARY

By design.

When synchronous and asynchronous redeem queues are enabled simultaneously, it is the curator's responsibility to maintain a sufficient liquidity buffer to satisfy both types of redemptions. The protocol intentionally does not reserve liquidity for asynchronous redemption requests, as synchronous redemptions are designed to have priority as long as adequate liquid assets are maintained.

Accordingly, operating both queue types safely relies on appropriate liquidity management rather than protocol-level reservation of assets for pending asynchronous batches.

M-3 Synchronous redemption loses accrued protocol fee on the burned shares

● MEDIUM

● ACKNOWLEDGED

DESCRIPTION

The time-based protocol fee is accrued on the share supply and settled only at report handling.

`ShareModule.handleReport` computes the fee on the current supply and mints it to the fee recipient before the queues process the report:

```
fees = feeManager_.calculateFee(
    address(this), asset, priceD18,
    shareManager_.totalShares() - shareManager_.activeSharesOf(feeRecipient_)
);
if (fees != 0) { shareManager_.mint(feeRecipient_, fees); }
feeManager_.updateState(asset, priceD18);
```

`src/modules/ShareModule.sol#L286-L303`

`calculateFee` accrues the protocol fee as `totalShares * protocolFeeD6 * (block.timestamp - lastTimestamp) / 365e6 days` (`src/managers/FeeManager.sol#L82`) - proportional to the supply present at report time and to the interval since the previous report.

The asynchronous `RedeemQueue` preserves this base: a redeemer's shares are locked at request time and burned only inside `RedeemQueue._handleReport` (`src/queues/RedeemQueue.sol#L239`), which runs after the fee is minted on the full supply - so the exiting shares still pay protocol fee for the interval they were held.

`SyncRedeemQueue.redeem` instead burns the caller's shares immediately (`src/queues/SyncRedeemQueue.sol#L129`), in a transaction between reports. At the next report those shares are no longer in `totalShares`, so the protocol fee that accrued on them over `[lastReport, redeem]` is never charged and is lost to the protocol. Severity is Medium: a systematic under-collection of protocol fee favoring synchronous redeemers over both the protocol and async redeemers; the magnitude is second-order (`protocolFeeD6 * interval-fraction * shares`) and zero while `protocolFeeD6 = 0`.

RECOMMENDATION

Settle the protocol fee accrued on the redeemed shares before burning them in

`SyncRedeemQueue.redeem` - e.g. trigger fee accrual on the current supply (mirroring the report path) at the start of `redeem`, so a synchronous exit pays the same protocol fee for its holding interval as an asynchronous one. Alternatively, document the divergence as an accepted trade-off of instant redemption.

CLIENT'S COMMENTARY

By design.

In practice, synchronous redemptions are expected to represent only a very small fraction of the vault's TVL, with the corresponding `dailyLimit` configured at well below 1% of total assets. As a result, the amount of protocol fees that is not accrued on shares redeemed through the synchronous queue between two reports is negligible.

We consider this an acceptable trade-off for providing instant redemptions while keeping the synchronous redemption path simple and gas-efficient.

2.4 Low

L-1 Global daily-limit bucket allows one holder to transiently block all redemptions

● LOW

● ACKNOWLEDGED

DESCRIPTION

The daily redemption limit is a single leaky bucket shared across all callers of a given queue, not a per-user allowance. Every redemption increments the same `usage` :

```
if (shares > _syncUsage()) {
    revert DailyLimitOverflow();
}
$.usage += shares;
```

`src/queues/SyncRedeemQueue.sol#L123-L126`

Because `redeem` is permissionless and instant, a single large holder can consume the entire day's redemption capacity in one transaction (bounded by `min(dailyLimit, liquidAssets)`), causing every other user's `redeem` call to revert with `DailyLimitOverflow` until the bucket decays linearly over time (`remainingDailyLimit`, [L40-L50](#)).

The impact is limited: the actor must own the shares being redeemed and pays the configured penalty and redeem fee on the full amount, so sustained griefing is self-financing and self-defeating, and the block is only transient (capacity refills continuously). The underlying accounting itself is conservative and correct - this is purely the fairness consequence of a global rather than per-account bucket.

RECOMMENDATION

Document that the daily limit is a single global bucket per queue so that operators size `dailyLimit` (and any off-chain UX expectations) with the transient first-come-first-served blocking behavior in mind.

CLIENT'S COMMENTARY

By design.

The daily redemption limit is intentionally implemented as a single global bucket shared across all users of the queue, rather than a per-user allowance. This behavior is expected and is part of the mechanism used to cap the protocol's aggregate synchronous redemption rate.

L-2 redeem has no user-side slippage protection while execution terms can change between signing and inclusion

● LOW

● ACKNOWLEDGED

DESCRIPTION

`SyncRedeemQueue.redeem` ([src/queues/SyncRedeemQueue.sol#L79](#)) is effectively a market order: the caller commits `shares` with no control over execution terms. While the transaction sits in the mempool, `setSyncRedeemParams` may raise `penaltyD6` (up to the 50% cap, so the user silently receives as little as half), a new oracle report may change `priceD18`, or `FeeManager` parameters may change. The async `RedeemQueue` needs no such guard (its price is fixed by a later report by design), but a `minAssets` bound is standard for a synchronous, exchange-like path.

Severity is Low: under the stablecoin `earnUSD*` oracle config (`timeout = 20h`, deviation $\leq 0.5\%$) the price vector is weak - a fresh report rarely lands in the mempool window and the move is small - so the realistic vector is a `penaltyD6` front-run. The price vector becomes live if the queue is paired with a faster or more volatile oracle.

RECOMMENDATION

Add a `minAssets` parameter (and optionally a `deadline`):

```
function redeem(
    uint256 shares,
    address receiver,
    uint256 minAssets
) external nonReentrant {
    ...
    if (assets < minAssets) revert SlippageExceeded(assets, minAssets);
```

CLIENT'S COMMENTARY

By design.

The synchronous redemption flow intentionally follows the same design as ERC-4626 and other vault implementations supporting instant withdrawals in this matter. It executes against the vault's current state at the time of transaction execution and does not provide user-side slippage protection.

L-3 **dailyLimit validation footguns: % 24 hours rejects natural values, dailyLimit = 0 silently bricks the queue, and lowering the limit freezes it**

● LOW

● ACKNOWLEDGED

DESCRIPTION

`SyncRedeemQueue._setSyncRedeemParams` validates `dailyLimit` (a share amount, the leaky bucket's per-24h capacity) with:

```
if (dailyLimit % 24 hours != 0) {
    revert InvalidDailyLimit();
}
```

`src/queues/SyncRedeemQueue.sol#L157-L159`

This requires the share amount to be a multiple of `86 400`, so natural values such as `1000e18` are rejected with `InvalidDailyLimit`, forcing operators to pick unintuitive numbers. The requirement is a precision guard for the integer decay (`Math.mulDiv(dailyLimit, timespan, 24 hours)`, [L47](#)), but `mulDiv` already bounds the error to dust, so the constraint buys nothing.

Conversely, `dailyLimit = 0` passes validation (`0 % 86 400 == 0`) and pauses the queue: `remainingDailyLimit` is then always `0`, so every `redeem` reverts `DailyLimitOverflow` ([L123](#)), with no dedicated error or event signalling the misconfiguration.

Finally, `usage` is not rescaled when `dailyLimit` changes. `_setSyncRedeemParams` decays `usage` at the old rate up to the change (`_syncUsage()` runs first, [L161](#)), then sets the new limit but leaves the residual `usage` untouched, so lowering the limit while usage is high freezes the queue for roughly `usage / newDailyLimit` days. This also breaks the parameter's own semantics: after a decrease the bucket can take several days to drain, so it is no longer a "daily" limit, and the "Maximum leaky-bucket capacity" description of `dailyLimit` no longer holds.

Severity is Low: each case requires a trusted-role configuration action (`SET_SYNC_REDEEM_PARAMS_ROLE`) and causes availability impact only.

RECOMMENDATION

Drop the divisibility requirement (or document it prominently); explicitly reject `dailyLimit == 0`, or document it as an intentional off-switch; and rescale `usage` proportionally on limit changes (`Math.mulDiv(usage, newLimit, oldLimit)`, or reset to `0` when `oldLimit == 0`), so lowering the limit does not overshoot into a multi-day freeze. The rescale truncates by at most 1 wei of shares, rounding usage down (in the protocol's favor), so it introduces no meaningful precision loss.

CLIENT'S COMMENTARY

By design.

The requirement for `dailyLimit` to be divisible by 24 hours is intentional. Without it, the calculation of the remaining daily limit would be subject to rounding, introducing precision loss over time on each block.

In practice, `dailyLimit` can be viewed as a more intuitive representation of an integer per-second redemption rate, scaled to a 24-hour period.

L-4 **penaltyD6 cap of 5e5 (50%) is excessively generous for a parameter that directly confiscates user value**

● LOW

● ACKNOWLEDGED

DESCRIPTION

`SyncRedeemQueue._setSyncRedeemParams` caps `penaltyD6` at `5e5` = 50%:

```
if (penaltyD6 > 5e5 || maxAge > 365 days) {  
    revert TooLarge();  
}
```

[src/queues/SyncRedeemQueue.sol#L151](#)

At the cap, a redemption burns the caller's gross `shares` (L129) but pays assets only on the penalty-reduced amount `sharesToRedeem = shares * (1e6 - penaltyD6) / 1e6` (L105, L112) - so a compromised or malicious holder of `SET_SYNC_REDEEM_PARAMS_ROLE` can legitimately confiscate up to half of the value of every synchronous redemption. Combined with the absence of slippage protection (Low #2), in-flight users have no defense against a parameter front-run. Expected operational values are fractions of a percent to single percents, so the cap provides no meaningful protection at its current level.

Severity is Low: the actor is trusted per the scope's threat model, and the finding concerns worst-case damage limitation.

RECOMMENDATION

Tighten the cap to an operationally plausible ceiling (e.g. `5e4` - `1e5`, i.e. 5-10%), and/or timelock `setSyncRedeemParams`.

CLIENT'S COMMENTARY

By design.

The 50% upper bound for `penaltyD6` is merely a sanity/adequacy check.

L-5 ETH sent directly to the queue (or a redemption with receiver == address(this)) is permanently stuck - no rescue method

● LOW

● ACKNOWLEDGED

DESCRIPTION

The queue exposes an unconditional payable `receive()` :

```
receive() external payable {}
```

[src/queues/SyncRedeemQueue.sol#L59](#)

It is required for native-ETH redemptions: `redeem` calls `vault_.callHook(assets)` , which sends ETH back to the queue via `Address.sendValue` ([src/modules/ShareModule.sol#L268](#) , [src/libraries/TransferLibrary.sol#L28-L29](#)) - an ETH transfer that would revert without a payable `receive()` - before `redeem` immediately forwards it to `receiver` ([L135](#)). The queue is thus meant to hold ETH only transiently within a single transaction and never between transactions.

However, `receive()` is unguarded, so it equally accepts arbitrary direct transfers, and the contract (including its `SyncQueue` base) has no sweep/rescue function - any such ETH is stuck forever. Similarly, `redeem` guards only `receiver == address(0)` ([L80](#)), not `receiver == address(this)` , so a redemption paid to the queue itself strands the payout with no recovery path. A finding of the same class (ETH can get stuck on StrategyCallForwarder) was raised and fixed in a companion vaults-wrapper audit.

Severity is Low: user error is required and the amounts are expected to be small.

RECOMMENDATION

Add a permissioned rescue method for surplus balances (since a correct redemption always leaves the queue at zero, any residual is safe to sweep), and/or reject `receiver == address(this)` in `redeem` . Additionally, since `receive()` is only ever meant to be reached by the vault sending ETH back during `callHook` , gate it to that single caller so stray direct transfers revert at the source:

```
receive() external payable {
    if (msg.sender != vault()) revert Forbidden();
}
```

CLIENT'S COMMENTARY

Acknowledged.

L-6 Under a prolonged oracle outage there is no exit path, and recovery requires two distinct oracle roles

● LOW

● ACKNOWLEDGED

DESCRIPTION

If the oracle stops submitting reports, synchronous redemption reverts with `StaleReport` once `maxAge` elapses (`src/queues/SyncRedeemQueue.sol#L99-L101`). No funds are stuck - shares remain active in users' wallets, so this is a pure availability loss. The asynchronous `RedeemQueue` (out of scope, noted for completeness) is worse: `redeem` locks shares immediately, pricing happens only on the next report, and there is no cancel function (unlike `DepositQueue.cancelDepositRequest`), so a user requesting an async exit during the outage loses access to those shares until reports resume.

Recovery is a two-role dependency. After a long gap the first submitted price will likely exceed the deviation thresholds and be flagged suspicious; `Oracle.submitReports` only pushes a report to the vault when it is not suspicious (`src/oracles/Oracle.sol#L116`), so a separate `ACCEPT_REPORT_ROLE` must call `acceptReport` (`src/oracles/Oracle.sol#L124-L140`) before any queue resumes. If the Oracle contract itself is broken, `ShareModule` stores the oracle address with no setter (`src/modules/ShareModule.sol#L324`), so replacement requires a proxy upgrade.

Severity is Low for this scope: availability only, funds are safe; the async no-cancel trap is an out-of-scope systemic concern.

RECOMMENDATION

Document the outage runbook (both `SUBMIT_REPORTS_ROLE` and `ACCEPT_REPORT_ROLE` are required for recovery); consider a `cancelRedeemRequest` for async requests not yet covered by a report (out of scope); and weigh the `maxAge` tension explicitly - small values DoS the sync path on any oracle hiccup, large values widen the stale-price window (Medium #1).

CLIENT'S COMMENTARY

Acknowledged.

L-7 `modifyVaultBalance` is called after the external asset transfer in `redeem` (checks-effects-interactions)

● LOW

● ACKNOWLEDGED

DESCRIPTION

`SyncRedeemQueue.redeem` settles the risk-manager accounting after transferring assets to the caller-controlled `receiver`:

```
vault_.callHook(assets); // vault -> queue
TransferLibrary.sendAssets(asset_, receiver, assets); // -> receiver
IVaultModule(address(vault_)).riskManager()
    .modifyVaultBalance(asset_, -int256(assets));
// accounting AFTER
```

[src/queues/SyncRedeemQueue.sol#L134-L136](#)

For native ETH (`Address.sendValue` forwards all gas) or an ERC-777-style token, `receiver` gains control while `RiskManager.vaultState.balance` is still overstated by `assets`. The ordering also deviates from `SyncDepositQueue.deposit`, which settles `modifyVaultBalance` before `callHook` ([src/queues/SyncDepositQueue.sol#L107-L108](#)).

No exploit path was found, so this stands as a CEI/defense-in-depth and consistency issue. The stale `vaultState.balance` is only read by `modifyVaultBalance` / `modifyPendingAssets`, and only their `change > 0` branch compares against `limit` ([src/managers/RiskManager.sol#L229](#)) - so a stale-high balance makes a re-entrant deposit more likely to hit `LimitExceeded`, never enabling an over-withdrawal. All redemption liquidity checks use real token balances via `getLiquidAssets`, which are already decremented by the time `sendAssets` runs. Severity is Low.

RECOMMENDATION

Move `riskManager().modifyVaultBalance(asset_, -int256(assets))` before `TransferLibrary.sendAssets`, mirroring the deposit queue, so internal accounting is settled before control can leave the protocol.

CLIENT'S COMMENTARY

Acknowledged.

L-8 maxAge has no lower bound tied to the oracle report cadence, so a below-timeout value causes a recurring self-inflicted DoS

● LOW

● ACKNOWLEDGED

DESCRIPTION

`SyncRedeemQueue._setSyncRedeemParams` validates `maxAge` only against `[1, 365 days]` ([src/queues/SyncRedeemQueue.sol#L151-L156](#)); it is never checked against the oracle's `timeout` (the minimum spacing between non-suspicious reports).

If an operator sets `maxAge < timeout` (e.g. a well-intentioned `maxAge = 1h` "for freshness"), synchronous redemption works only for `maxAge` after each report and is then DoS'd for the remaining `timeout - maxAge` of every cycle - a permanent, recurring outage with no configuration-time error to signal it.

Severity is Low: trusted-role misconfiguration, availability impact only (async redemption remains available).

RECOMMENDATION

Document the required relationship (`maxAge` must exceed the oracle's non-suspicious report cadence with margin) and consider validating a sane floor at configuration time - e.g. read `oracle().securityParams().timeout` in `_setSyncRedeemParams` and require `maxAge >= timeout` (a sanity floor, not a guarantee, since `securityParams` can change later).

CLIENT'S COMMENTARY

Acknowledged.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[!\[\]\(9f3852d68d41e1e95bc4ec10e81aba4b_img.jpg\)](#) [!\[\]\(bdb0f3befa0da67ab318d6edc913daee_img.jpg\)](#) [!\[\]\(bf8000641e0d351c7a9edc2d7fb94a96_img.jpg\)](#) [!\[\]\(bc6b00183e2904c191047813e921a4e5_img.jpg\)](#)